National University of Computer and Emerging Sciences

CL1004
Object Oriented Programming
Lab

Lab Instructor	**Ms. Areesha Faisal**
Semester	**Spring 2026**
Email	**areesha.faisal@nu.edu.pk**
Office	**Ground Floor, SFC Building**

Objective:

- Semester End Project Report

1. Project Title:

Project Name: Object-Oriented Chess Game

Course: Object-Oriented Programming (OOP) Semester Project

Date: May 2026

2. Group Members:

Name	Roll Number	Contribution
Muhammad Dawood	25F-3002	Class Queen, Chess Board, Game Engine
Abdullah Matloob	25F-3079	Base Class, Bishop, King
Faisal Ali	25F-3078	Class Pawn, Rook, Knight

3. Problem Statement:

The main objective of this project was to create a functional, Command Line Interface (CLI) based Multiplayer Chess Game that contained all the possible rules of a standard Chess Game using the concepts of structures in C++ Programming Language.

The challenge:

- Preventing invalid moves
- Handling paths
- Managing player’s turn
- Detecting the end-game condition

4. Problem Solution:

We used the concept of OOP to solve this problem:

- **Abstraction:**
    - Created Abstract Base Class (ChessPeice) with Pure Virtual Function
        - This acts as a blueprint for all pieces

- **Inheritance:**
    - Pieces (Pawn, Rook, Knight, Bishop, King, Queen) inherited from Base Class (ChessPeice)
        - reused the attributes while defining own move logic

- **Polymorphism:**
    - The ChessBoard class stores pointers to the base class (a grid of 8x8)
        - ValidMove function checks the validity of move irrespective of the peice

- **Encapsulation:**
    - Board state and Turn are hidden within ChessBoard and ChessEngine classes

- **Composition:**
    - The ChessEngine Class contains ChessBoard object, making a "has-a" relation

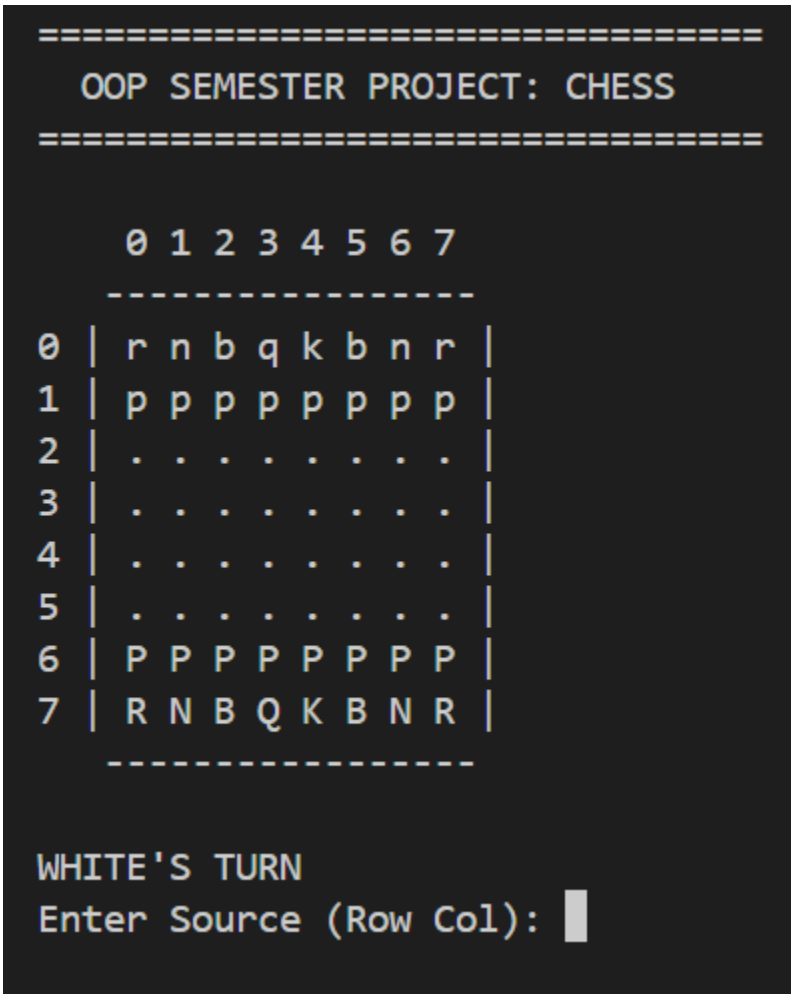
5. Working & Execution

Logical Flow:

1. **Initialization:**
    - ChessBoard Constructor calls initializePieces function which allocate every piece to its staring position
2. **The Game Loop:**
    - ChessEngine has a while(true) loop
    - Displaying Current Board in 2D Co-ordinates and prompt user to enter move
3. **Validation:**

    - Check if not attacking your own team
    - isValidMove is called for each piece to check path validation
4. **Execution:**
    - If move is valid, position is changed
    - If an enemy piece is there, its memory is deleted
5. **Termination:**
    - If King is captured, the game displays "Game Over" message

6. Screenshots



Starting

```
    0 1 2 3 4 5 6 7
   -----------------
0 | r n b q k b n r |
1 | p . p p p p p p |
2 | . . . . . . . . |
3 | . p . . . . . . |
4 | P . . . . . . . |
5 | . . . . . . . . |
6 | . P P P P P P P |
7 | R N B Q K B N R |
   -----------------
```

First Move

```
WHITE'S TURN
Enter Source (Row Col): 4
0
Enter Target (Row Col): 3
1

    0 1 2 3 4 5 6 7
   -----------------
0 | r n b q k b n r |
1 | p . p p p p p p |
2 | . . . . . . . . |
3 | . P . . . . . . |
4 | . . . . . . . . |
5 | . . . . . . . . |
6 | . P P P P P P P |
7 | R N B Q K B N R |
   -----------------
```

Captured Enemy's Pawn

```
    0 1 2 3 4 5 6 7
   -----------------
0 | r n b q . b n r |
1 | p . p p . p p p |
2 | . . . . k . . . |
3 | . P . Q . . . . |
4 | . . . . . . . . |
5 | . . . . . . . . |
6 | . P P . P P P P |
7 | R N B . K B N R |
   -----------------

WHITE'S TURN
Enter Source (Row Col): 3
3
Enter Target (Row Col): 2
4

GAME OVER! WHITE CAPTURED THE KING!
```

Queen Captured the King